

Docker Compose

لما احي اعمل **Implementation** = هو تنفيذ أو كتابة الكود نفسه أو **Software Architecture**

- في 3 انواع من الـ **Software Architecture**

- **Monolithic**
- **Microservice**
- **Service-criented Architecure (SOA)**

هنتكلم دلوقتي عن الـ **Monolithic** & **Microservice**

أي الفرق بين الـ **Monolithic** & **Microservice**

1. **Monolithic**

- يعني الـ **App** بتاعي كله عبارة عن **Single Unit**
- كل الـ **Service** في نفس المكان
- مكتوبه بلغة واحده للكل
- لو عندي موقع **Ecommerce** بالطبع هيكون فيه **Process** كتيره جدا للدفع والتحميل وتسجيل الدخول والاسترجاع -----
- كل الـ **Process** دول في مكان واحد بلغة واحده

2. **Microservice**

- بتفصل كل **Service** لوحدها بشكل مستقل
- كل **service** اقدر اخليها بلغة برمجيه مختلفه عن التانيه
- كل **Service** ليها الـ **Container** بتاعها
- ممكن يكون عندي مئات الـ **Containers**

مزايا الـ **Monolithic**

1. سهل في عملية الـ **Deploy** مكتوب بلغة برنجيه واحده بالتالي كل الـ **Requirement** موحد
2. التكلفه اقل

مشاكل الـ **Monolithic**

1. الـ **App** عبارة عن **Single Unit** يعني لو حصل أي **crash** الـ **App** كله هيقع ويتعطل بكل الـ **Functions**
2. أي **Update** هيكون متعب جدا ممكن بكل بساطه اعمل **update** في **service** معينه يحصل مشكله في مكان تاني وبالتالي حاجات كتير تتعطل
3. كتابة الـ **App** كله بلغة واحده مميز وسهل
 - لكن هيققل التكامل طبعا في لغات كتير جدا وكل لغة متخصصه في حاجه في لغة بتركز على الأمان ولغة بتركز على الأداء
 - ولغة بتركز على المرونه لذلك الاعتماد على لغة واحده هيققل التكامل
 - ممكن اللغة دي يكون فيها **features** مش موجوده انت محتاجها

مزايا الـ Microservice

- فصل كل الـ Service وجعلها مستقلة هوصل لأعلى مستوى في الـ Stability (الاستقرار)
- لما يكون الـ Service مستقلة هقدر اشوف أفضل لغة برمجه لكل Service
- لو حصل أي مشكله في أي service بتكون مستقلة بنفسها مش بتسمع في كل الـ App بالتالي لو في مشكله في الـ Service الـ payment الـ App هيشغل بشكل طبيعي جدا لكن الـ Payment هي اللي مش شغاله
- بتوفرلي الـ (Scalability) يعني قابل للتوسع لو في أي ضغط على service
- سهولة إضافة الـ Service
- لو حصل مشكله في أي Service وكانت depended مع service تانيه مفيش أي مشكله لأن الـ App مش هيقع

عيوب الـ Microservice

- هحتاج لكل لغة تيمم كامل
- مكلف جدا
- ممكن يكون عندي مئات والاف الـ containers وممكن ملايين ساعتها هتكون ادارته صعبه جدا وهيكون بشكل Manual

امتى اختار الـ Monolithic OR Microservice

- على حسب حجم الـ App الـ Service اللي هيقدمها لو هقدم Service كتير اكيد Microservice
- لو الـ App بقدم فيه service او two service وفالمستقبل هيحصل تطور يبقى هستعمل الـ Microservice
- في مرحلة الـ Planing بيتم تحديد الـ Project هيكون (Monolithic OR Microservice)

الـ DevOps مبني على الـ Microservice

كان في عيب مهم جدا في الـ Microservice

- ممكن يكون عندي مئات والاف الـ containers وممكن ملايين ساعتها هتكون ادارته صعبه جدا وهيكون بشكل Manual
- الـ Docker Compose جاي يحل المشكله دي عشان يخلي عملية الـ Manage تكون بشكل Automatic

الـ Docker Compose عبارة عن Tool مستقلة من خلالها — Create Configuration File

- جوا الـ File بـ Defining كل الـ Containers الـ Related بالـ App

يعني أي Defining

- إنك بتحدد مواصفات وإعدادات الكونتير قبل ما يشتغل

- يعني بتقول لـ Docker :

• الـ Container ده هيشغل إزاي؟

• هستخدم أنهي Image ؟

• هيفتح أنهي Port ؟

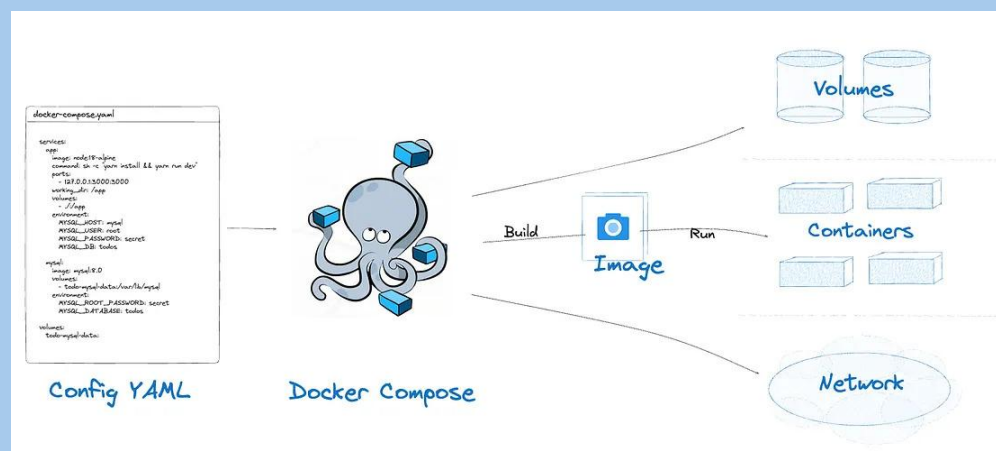
• هيحفظ الـ Data فين؟

• هيتصل بأني Network ؟

عملية الـ Defining بتحصل بطريقتين رئيسيتين:

1. docker run

2. Docker Compose



3.

- الأخطبوط عنده 8 أذرع، وكل ذراع يقدر يتحرك ويتحكم فيه بشكل مستقل بفضل الشبكة العصبية الموزعة في أذرع.

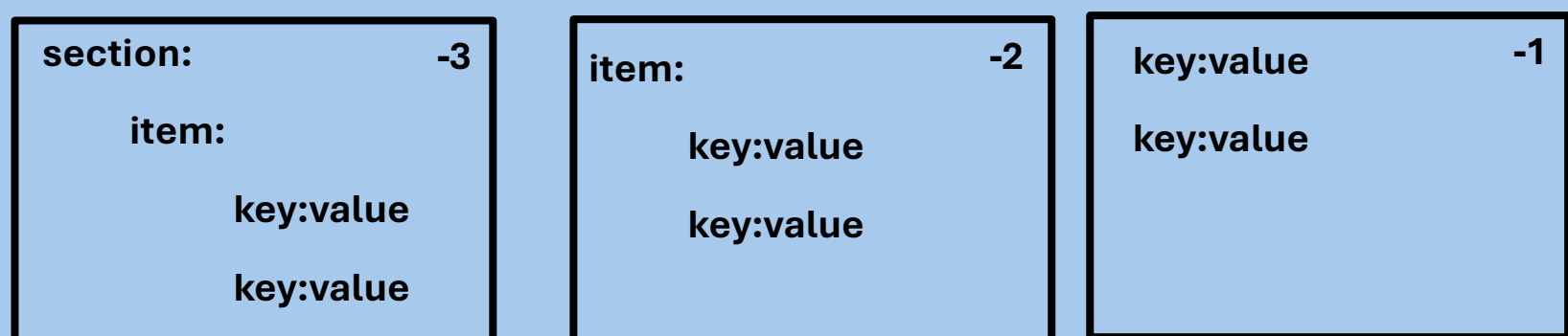
بنفس الفكرة، **Docker Compose** كمدير مركزي لمجموعة من الخدمات الـ (**Containers**) ، فبيقدر يشغلها ويوقفها

ويربطها ببعض ويدير الشبكات (**Networks**) ووحدات التخزين (**Volumes**) من خلال ملف واحد **docker-compose.yml**.

كل **Container** بيعمل وظيفته بشكل مستقل، لكن **Docker Compose** ينسق العمل بينهم كلهم كأنهم نظام واحد متكامل.

File الـ Compose بيتكتب بـ yml Format

في 3 طرق لكتابة الـ yml



الـ Docker Compose بيستخدم الطريقة الـ 3

لازم يكون الـ Compose File أخرة .yaml OR .yml

لازم يكون اسم الـ File ← compose OR docker-compose

```
yousef@DevOpsVM:~/project$ vim docker-compose.yaml
yousef@DevOpsVM:~/project$ vim docker-compose.yml
yousef@DevOpsVM:~/project$ vim compose.yml
yousef@DevOpsVM:~/project$ vim compose.yaml
```

- أي شكل من الأشكال دي صح

```
services:
  login:
    container_name: login_cont
    image: nginx:latest
    ports:
      - "8080:80"
    networks:
      - "net1"
      - "net2"
    volumes:
      - login_data:/usr/share/nginx/html
    environment:
      var1: docker
      var2: devops
    depends_on:
      - market

  market:
    container_name: market_cont
    image: myapp:v1
    ports:
      - "9090:80"
    networks:
      - "net2"
    volumes:
      - market_data:/usr/local/apache2/htdocs
    depends_on:
      - payment

  payment:
    container_name: payment_cont
    image: httpd:latest
    networks:
      - "net2"
    volumes:
      - payment_data:/usr/local/apache2/htdocs

networks:
  net1:
  net2:

volumes:
  login_data:
  market_data:
  payment_data:
```

لو دقت بكل بساطه الموضوع كله عبارة عن **format** نفس الأوامر الأساسية بس بطريقة مختصرة واسهل وأفضل

بنعمل **container** وبنعمل **Network** وهكذا كأنك مستعمل **docker run**

[شرح الـ **yml format**]

- **services**
 - دي الكلمة الأساسية (root key) في ملف **docker-compose.yml** تحتها بتحت كل الـ **containers**
 - **login**
 - الـ **item**
 - ده اسم السيرفيس (service name) انت اللي بتسميه
 - **container_name**
 - الـ **key**
 - بيحدد اسم الكونتير اللي هيتعمل
 - **cont_login**
 - الـ **Value**
 - **ports**
 - لو الـ **key** بيتكتب فيه اكر من **value** بيتكتب بالشكل دا
 - هنا عندك (array) **list** ماش **value** واحدة
 - **depends_on**
 - معناها: الـ **Service** دي بتعتمد على **Service** تانية
 - **networks**
 - بتحدد الشبكة اللي الكونتير هيشغل عليها
 - **volumes**
 - لربط (data persistence) **storage**
 - هتلاحظ انهم متعرفين تحت + لو مش معمول هيتم انشائه بشكل تلقائي
 - الـ **docker compose** بيمشي بالأولويه واي حاجه قبل انشاء الـ **container** ليها الأولويه
-

ازاى اشغل الـ Docker Compose

في عندنا امرين

1. docker compose

2. docker-compose

وبتعرف تشغل انهى من خلال امر

```
yousef@DevOpsVM:~/project$ docker compose version
Docker Compose version v5.0.1
```

لازم أكون واقف في مكان الـ file docker compose

لو اول مره اعمل Run لازم ابدأ بكتابة امر

```
yousef@DevOpsVM:~/project$ sudo docker compose up -d
[sudo] password for yousef:
WARN[0000] No services to build
[+] up 8/8
✓ Network project_net2          Created
✓ Network project_net1          Created
✓ Volume project_payment_data   Created
✓ Volume project_login_data     Created
✓ Volume project_market_data    Created
✓ Container payment_cont        Created
✓ Container market_cont         Created
✓ Container login_cont          Created
yousef@DevOpsVM:~/project$ |
```

عشان نشوف اللي حصل

```
yousef@DevOpsVM:~/project$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
64405f8671b1   nginx:latest   "/docker-entrypoint...." 2 minutes ago Up 2 minutes  0.0.0.0:808
0->80/tcp, [::]:8080->80/tcp
login_cont
d69d8a680891   myapp:v1       "httpd-foreground"      2 minutes ago Up 2 minutes  0.0.0.0:909
0->80/tcp, [::]:9090->80/tcp
market_cont
0401bf410993   httpd:latest   "httpd-foreground"      2 minutes ago Up 2 minutes  80/tcp
payment_cont
```

```
yousef@DevOpsVM:~/project$ sudo docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
b174bbfb4097   bridge    bridge    local
0949f201ad22   host      host      local
b30ea710d25c   none      null      local
7bcc2dbb6f59   pri_net   bridge    local
387d250f2ace   project_net1 bridge    local
1d30b36143ed   project_net2 bridge    local
6228895e324f   pub_net   bridge    local
```

```
yousef@DevOpsVM:~/project$ sudo docker volume ls
DRIVER    VOLUME NAME
local     appdata
local     project_login_data
local     project_market_data
local     project_payment_data
```

- هتلاحظ كلمة **project** دي معناها ال **directory** اللي اللي فيه الـ **compose**

لو عايز اظهر فقط الـ **containers** اللي اتعملت من خلال الـ **compose**

```
yousef@DevOpsVM:~/project$ sudo docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
login_cont	nginx:latest	"/docker-entrypoint..."	login	55 minutes ago	Up 55 minutes
market_cont	myapp:v1	"httpd-foreground"	market	55 minutes ago	Up 55 minutes
payment_cont	httpd:latest	"httpd-foreground"	payment	55 minutes ago	Up 55 minutes

لو عايز اوقف الـ **containers**

```
yousef@DevOpsVM:~/project$ sudo docker compose stop
```

```
[+] stop 3/3
```

- ✓ Container login_cont Stopped
- ✓ Container market_cont Stopped
- ✓ Container payment_cont Stopped

لو عايز اشغلهم تاني

```
yousef@DevOpsVM:~/project$ sudo docker compose start
```

```
[+] start 3/3
```

- ✓ Container payment_cont Started
- ✓ Container market_cont Started
- ✓ Container login_cont Started

- هتلاحظ لما شغلهم استعمل الأولويه عشان الـ **login** بتعتمد على الـ **market** والـ **market** بيعتمد على الـ **payment**

لو عايز اشغل **container** معين

```
yousef@DevOpsVM:~/project$ sudo docker compose start market
```

```
[+] start 2/2
```

- ✓ Container payment_cont Started
- ✓ Container market_cont Started

هتلاحظ ان هو شغل الـ **payment** لأن الـ **market** بتعتمد على الـ **payment**

لو عايز احذف كل حاجه خاصة بالـ **compose**

```
yousef@DevOpsVM:~/project$ sudo docker compose down
```

```
[+] down 5/5
```

- ✓ Container login_cont Removed
- ✓ Container market_cont Removed
- ✓ Container payment_cont Removed
- ✓ Network project_net1 Removed
- ✓ Network project_net2 Removed

- هتلاحظ انه حذف كل حاجه ما عدا الـ **data** لأنك ممكن تكون محتاجها

- هو حافظ على الـ **data**

لو عايز اشغلهم تاني بكل بساطه

```
yousef@DevOpsVM:~/project$ sudo docker compose up -d
WARN[0000] No services to build
[+] up 5/5
✓ Network project_net2      Created
✓ Network project_net1      Created
✓ Container payment_cont     Created
✓ Container market_cont      Created
✓ Container login_cont       Created
```

- ال **images** موجودين الملف جاهز مجرد ان انا أنشأت ال **containers** مره ثانيه
- حافظنا على ال **data** بشكل مستقل
- سهولة استرجاع أي شيء لو ال **server** حصله حاجه